SENG401 – GRADUATION PROJECT

Software Requirements Specification Document

By Sinem Coşkun, Al-baraa Abdullah Abdullah Abdullah Al-yadumi

TABLE OF CONTENTS

1. Introduction 1
1.1 Purpose 1
1.2 Scope 2
1.3 Definitions, Acronyms, and Abbreviations 3
1.4 References 3
1.5 Document Overview 4
2. Overall Description 4
2.1 Product Perspective 4
2.2 Product Functions 5
2.3 User Classes and Characteristics 5
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 6
2.6 Assumptions and Dependencies 7
3. System Features (Functional Requirements) 7
3.1 Edge Honeypot Operation 7
3.2 Log Collection and Aggregation 8
3.3 MQTT-Based Log Transmission 8
3.4 Central Log Storage (SQLite) 9
3.5 Data Preprocessing and Normalization 9
3.6 Feature Extraction 10
3.7 Statistical Tunneling Detection 10
3.8 Machine-Learning-Based Attack Classification 11
3.9 Time-Series Storage of Analysis Results (InfluxDB) 12
3.10 Visualization and Reporting 12
4. External Interface Requirements 13
4.1 User Interfaces 13
4.2 Hardware Interfaces 13
4.3 Software Interfaces 14
4.4 Communication Interfaces 15
5. Non-Functional Requirements 15
5.1 Performance Requirements 15

5.2 Reliability Requirements....16
5.3 Security Requirements....16
5.4 Usability Requirements....16
5.5 Maintainability Requirements....17
5.6 Extensibility Requirements....17
5.7 Portability Requirements....17
6. Other Requirements....17
6.1 Ethical and Privacy Considerations....17
6.2 Monitoring and Logging Requirements....18
6.3 Data Handling and Output Requirements....18
7. Appendices....19
7.1 Appendix A — Glossary....19
7.2 Appendix B — Sample Logs (Description)....20
7.3 Appendix C — Feature List....20
7.4 Appendix D — Evaluation Metrics....21
7.5 Appendix E — Project Schedule Overview....22
7.6 Appendix F — System Architecture Diagram....23

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe the software requirements of the project titled:

"Detecting Protocol Tunneling Activities and Cyber Attacks with Machine Learning-Based IoT Honeypot."

The project aims to develop a hybrid analysis system that runs on a low-resource IoT device (Raspberry Pi) and:

- Collects real attack traffic through honeypot systems configured on the Raspberry Pi,
- Detects DNS and HTTP protocol tunneling activities using statistical, entropy-based analysis, and
- Detects and classifies various cyber attack types (such as brute force, port scanning, malware distribution and bot/human behavior) using machine learning techniques.

This SRS defines what the system shall do, which data it shall process, which outputs it shall produce and which constraints it shall operate under, so that the implementation and evaluation can follow a clear specification.

1.2 Scope

The proposed system is an IoT honeypot–based monitoring and analysis platform built around a Raspberry Pi 4 Model B device.

Within the scope of the project:

- A Raspberry Pi is configured as an edge device exposed to the Internet through a guest network and a DMZ structure, without being connected to a production network.
- On this device, three open-source monitoring systems are installed:
    - Cowrie for SSH attack logging,
    - Honeytrap for HTTP and FTP service emulation,
    - Suricata as an IDS analyzing general network traffic.
- These systems generate log files that are transmitted to a central analysis computer using the MQTT protocol, where the logs are stored in a SQLite database.
- In the central environment, the data is processed in Python, and feature extraction is applied to obtain measurable traffic characteristics such as:
    - session duration,
    - query frequency,

    - character entropy,
    - URI length,
    - IP address diversity,
    - connection intensity.
- Two complementary analysis approaches are used:
    1. Statistical / Entropy-based tunneling detection using Shannon entropy and sliding-window analysis focused on DNS and HTTP protocol tunneling.
    2. Machine-learning-based attack analysis, where brute-force attacks, port scans, malware download attempts and bot–human behavior are classified using supervised or anomaly-detection models.
- The main evaluation metrics are accuracy, false positive rate and F1-score. For DNS and HTTP tunneling, an accuracy of at least 80% is targeted; for attack classification, classifiers with F1-score ≥ 80% are aimed.
- After the analysis, results are stored in InfluxDB, an open-source time-series database, and visualized with open-source visualization tools such as Grafana and Apache Superset, including:
    - tunneling detections,
    - classified attacks with timestamps,
    - originating countries of attacks,
    - bot vs. human behavior.
- At the end of the project, a local attack dataset, the analysis code and documentation will be shared as open source on GitHub, and the results will be used to produce academic publications.

The project is planned as an 8-month study, and the system is designed not as an active defense or prevention mechanism, but as a research and monitoring platform.

1.3 Definitions, Acronyms, and Abbreviations

- **Anomaly Detection** – Identifying patterns that deviate from normal system behavior.
- **Attack Classification** – Assigning detected events to categories such as brute force, port scanning, or bot activity.
- **Cowrie** – A medium-interaction SSH/Telnet honeypot capturing attacker behavior.
- **DMZ (Demilitarized Zone)** – A network segment that isolates publicly reachable devices from the internal network.
- **DNS/HTTP Tunneling** – Hiding data within DNS or HTTP traffic to bypass security controls.
- **Entropy (Shannon Entropy)** – A statistical measure of randomness used for tunneling detection.

- **Feature** – A numerical attribute extracted from logs (e.g., URI length, entropy score).
- **Grafana** – A visualization tool for time-series dashboards.
- **Honeypot** – A decoy system designed to attract and log malicious activities.
- **Honeytrap** – A low-interaction honeypot simulating common network services.
- **InfluxDB** – A time-series database used to store analysis results.
- **IoT (Internet of Things)** – A network of connected physical devices capable of communication.
- **Machine Learning (ML)** – Algorithms that learn from data to detect or classify attack behaviors.
- **MQTT** – A lightweight publish/subscribe protocol used to transfer logs.
- **SQLite** – A lightweight database storing raw logs and extracted features.
- **Sliding Window** – A method for evaluating entropy over sequential segments of data.
- **Suricata** – An IDS generating network security alerts and packet-level logs.
- **Time-Series Database** – A database optimized for timestamped data.

1.4 References

The following scientific and technical sources support the theoretical foundation and design decisions of the system:

[1] F. Al-Turjman et al., "Federated Learning for IoT Security and Privacy," *Future Internet*, 2025.
[2] M. Kaur & S. Singh, "IoT Network Security Threats and Countermeasures," *Computers*, 2025.
[3] R. N. Rao, "AI-Driven Detection of IoT Attacks using Honeypot Data," *PMC*, 2024.
[4] A. Ahmed et al., "A Honeypot-Driven Approach to IoT Attack Detection," *IET Smart Cities*, 2024.
[5] R. D. Venkat Goundla, "Analysis of IoT Honeypots for Network Security," *Applied Sciences*, 2025.
[6] V. G. Goundla, "Optimizing IoT Honeypots with Machine Learning," *NCI Repository*, 2024.
[7] D. Ivanov, "Securing Military Embedded Systems: Three Challenges," *Military Embedded Systems*, 2024.
[8] Quest Defense, "Evolution of Embedded Systems in Aerospace & Defense," *Defense Insights*, 2024.
[9] X. Huang & Y. Huo, "A Review of Tunnel Detection on Multilayer Protocols," *Applied Sciences*, 2023.
[10] Alve, "Smart IoT Security: Lightweight Machine Learning Techniques," *arXiv*, 2025.
[11] A. Bhandari et al., "Distributed Deep Learning for IoT Cyber-Attack Detection," *Electronics*, 2023.
[12] T. Salat et al., "DNS Tunnelling, Exfiltration and Detection in Cloud Environments," *Sensors*, 2023.
[13] F. Üstebay, "Enhancing Zero-Day Attack Detection in IoT Networks via Isolation," *Electrica*, 2025.

1.5 Document Overview

This SRS is structured as follows:

- **Section 2 — Overall Description:**
  Provides the system context, architecture perspective, user characteristics, constraints, and operating environment.
- **Section 3 — System Features:**
  Defines detailed functional requirements for each subsystem.
- **Section 4 — External Interface Requirements:**
  Describes user interfaces, hardware interfaces, software interfaces, and communication interfaces.
- **Section 5 — Non-Functional Requirements:**
  Outlines performance goals, security requirements, usability, maintainability, and other critical constraints.
- **Section 6 — Other Requirements:**
  Includes ethical considerations, monitoring needs, and data-handling requirements.
- **Section 7 — Appendices:**
  Contains the glossary, sample logs, and feature definitions.

2. Overall Description

2.1 Product Perspective

The system is designed as a complete IoT honeypot–based monitoring and analysis platform running on a Raspberry Pi device and connected to a central analysis environment. It operates in a real-world network setup by exposing the Raspberry Pi through a DMZ and guest network, ensuring that it receives genuine attack traffic.

The system integrates three major components:

1. **Edge Honeypot Layer** (on Raspberry Pi):
   - Cowrie (SSH honeypot)
   - Honeytrap (HTTP/FTP emulation)
   - Suricata (IDS for network traffic inspection)
2. **Data Transfer and Storage Layer:**
   - MQTT for lightweight log transmission
   - SQLite for storing all raw logs and extracted features
3. **Analysis and Visualization Layer:**
   - Statistical tunneling detection (Shannon entropy + sliding window)
   - Machine-learning-based attack classification
   - InfluxDB for storing analysis results
   - Grafana / Apache Superset for visualization

2.2 Product Functions

In general, the system performs the following functions:

- **Capture real attack traffic** through honeypots running different protocols.
- **Record attacker interactions** including commands, session metadata, and network packets.
- **Transfer log data** to the central system using MQTT.
- **Store logs** in a structured form in SQLite.
- **Extract meaningful traffic features**, such as entropy, URI length, session duration, and connection intensity.
- **Detect DNS/HTTP tunneling** using entropy-based statistical methods.
- **Classify cyber attacks** (brute force, port scan, malware download, bot/human).
- **Store analysis results** in a time-series database (InfluxDB).
- **Visualize analysis outputs** through dashboards (Grafana / Superset).
- **Provide evaluation metrics** such as accuracy, false positive rate, and F1-score.

2.3 User Classes and Characteristics

The system is intended for the following user groups:

- **Researcher / Analyst:**
  Uses the system to examine attack patterns, tunneling activities, and ML results.
- **System Maintainer / Developer:**
  Installs honeypots, manages MQTT/DB setup, updates code, and maintains the Raspberry Pi.

Both user types are expected to have basic knowledge of network security, Linux environments, and data analysis.

2.4 Operating Environment

The system operates in the following hardware and software environments:

Hardware

- Raspberry Pi 4 Model B (4GB RAM)
- Home/office router providing DMZ and guest network structure
- Central computer/server used as the analysis environment

Network Setup

- Raspberry Pi exposed to the internet via Guest Network + DMZ
- Direct connection to the internet to capture external real-world attacks

Software

- Cowrie, Honeytrap, Suricata honeypot/IDS tools
- MQTT broker
- SQLite database
- Python (Pandas, Scikit-learn) for analysis, preprocessing, and ML
- InfluxDB for time-series storage
- Grafana / Apache Superset for visualization

2.5 Design and Implementation Constraints

The system must operate under the following constraints:

- **Low-resource hardware:** Raspberry Pi has limited CPU, memory, and storage.
- **Real attack exposure:**
  The honeypot is directly exposed to the internet; configuration must avoid impacting any production network.
- **Log volume variability:**
  Real attack traffic may fluctuate; logs depend on attacker activity.
- **Dependency on open-source tools:**
  The system relies entirely based on open-source software (Cowrie, Honeytrap, Suricata, MQTT, SQLite, InfluxDB, Grafana, Superset).
- **Strict evaluation targets:**
  DNS/HTTP tunneling detection: ≥ 80% accuracy
  Attack classification: ≥ 80% F1-score

2.6 Assumptions and Dependencies

The system is based on the following assumptions:

- The Raspberry Pi will successfully receive sufficient external attack traffic once exposed to DMZ/Guest network.
- MQTT communication between the Raspberry Pi and central computer remains stable.
- SQLite storage is sufficient for the volume of logs produced during the project.
- Statistical and machine learning analyses will be performed using Python on the central machine.
- Visualization tools (Grafana/Superset) will properly connect to the InfluxDB instance.
- In case insufficient real-world data is collected, public datasets (CTU-13, CIC-IDS 2017) may be used as backup for training/testing, as stated in the risk management section.

3. System Features (Functional Requirements)

3.1 Edge Honeypot Operation

Description

The Raspberry Pi device runs honeypots (Cowrie, Honeytrap) and an IDS (Suricata) to capture actual attack traffic.

Inputs

- Incoming attack traffic from the internet via DMZ / guest network.
- Attacker commands, sessions, HTTP/FTP interactions, network packets.

Process / Behavior

- The honeypots interact with attackers and record their actions.
- Suricata inspects network packets and produces alerts/logs.

Outputs

- Log files containing timestamped attacker activities, commands, sessions, and alerts.

Functional Requirements

ID	Description
FR-1.1	The system shall run Cowrie to log SSH-based attacks and attacker commands.
FR-1.2	The system shall run Honeytrap to emulate HTTP/FTP services and collect associated attack traffic.
FR-1.3	The system shall run Suricata to generate IDS alerts and inspect network traffic.
FR-1.4	The system shall timestamp and store all attacker interactions in log files.
FR-1.5	The Raspberry Pi shall operate in DMZ/guest network mode to capture real external traffic.

3.2 Log Collection and Aggregation

Description

Log files produced by Cowrie, Honeytrap, and Suricata are collected and prepared for transmission.

Inputs

- Raw logs from the edge honeypots.

Process / Behavior

- All logs are gathered, combined if necessary, and queued for MQTT transmission.

Outputs

- Standardized log entries for MQTT transfer.

Functional Requirements

ID	Description
FR-2.1	The system shall collect logs from Cowrie, Honeytrap, and Suricata.
FR-2.2	The system shall merge or organize logs to ensure they can be transmitted through MQTT.
FR-2.3	Each log entry shall include the required metadata (timestamp, source, session data, etc.).

3.3 MQTT-Based Log Transmission

Description

The Raspberry Pi sends collected log data to the central analysis computer.

Inputs

- Logs from Section 3.2.

Process / Behavior

- Logs are published to MQTT topics.
- The central server subscribes and receives the logs.

Outputs

- Logs delivered to the central analysis machine.

Functional Requirements

ID	Description
FR-3.1	The system shall transmit logs to the central server using the MQTT protocol.
FR-3.2	The system shall ensure uninterrupted log forwarding while the honeypot is active.
FR-3.3	The system shall retry transmission in case of temporary connectivity issues.

3.4 Central Log Storage (SQLite)

Description

All incoming logs are stored in a structured SQLite database.

Inputs

- Log entries received via MQTT.

Process / Behavior

- Logs are parsed, validated, and inserted into SQLite tables.

Outputs

- Structured datasets ready for preprocessing, feature extraction, and analysis.

Functional Requirements

ID	Description
FR-4.1	The system shall store all received log data in a SQLite database.
FR-4.2	The system shall create tables corresponding to each log type or analysis requirement.
FR-4.3	The system shall validate each entry (e.g., formatting, required fields) before storage.

3.5 Data Preprocessing and Normalization

Description

Raw logs are cleaned, converted, and prepared for feature extraction.

Inputs

- Log entries stored in SQLite.

Process / Behavior

- Extract required fields, convert formats, remove duplicates, normalize values.

Outputs

- Clean and structured data ready for feature extraction.

Functional Requirements

ID	Description
FR-5.1	The system shall preprocess raw logs to extract required fields.
FR-5.2	The system shall remove corrupted, incomplete, or invalid records.
FR-5.3	The system shall normalize numerical features as required for analysis.

3.6 Feature Extraction

Description

The system extracts measurable traffic characteristics from logs to support tunneling detection and ML classification.

Inputs

- Preprocessed data.

Process / Behavior

- Calculate independent variables such as:
    - session duration
    - query frequency
    - character entropy
    - URI length

- IP address diversity
- connection intensity

Outputs

- Dataset of numerical features per session, query, or event.

Functional Requirements

ID	Description
FR-6.1	The system shall compute entropy metrics (Shannon entropy).
FR-6.2	The system shall extract all defined features (session duration, URI structure, connection intensity, etc.).
FR-6.3	The system shall store extracted features in SQLite for later analysis.

3.7 Statistical Tunneling Detection

Description

DNS and HTTP traffic is analyzed using entropy and sliding-window analysis to detect tunneling activity.

Inputs

- Extracted features related to DNS/HTTP traffic.

Process / Behavior

- Compute sliding-window entropy trends.
- Compare entropy against predefined thresholds.
- Mark traffic as “normal” or “tunneling.”

Outputs

- Tunneling detection decisions.

Functional Requirements

ID	Description
FR-7.1	The system shall apply Shannon entropy to DNS and HTTP traffic.
FR-7.2	The system shall apply sliding-window analysis to track entropy changes over time.
FR-7.3	The system shall classify traffic as tunneling or normal based on entropy thresholds.
FR-7.4	The system shall aim for ≥ 80% detection accuracy.

3.8 Machine-Learning-Based Attack Classification

Description

ML models classify different cyber attack behaviors using labeled data.

Inputs

- Feature dataset produced in Section 3.6.

Process / Behavior

- Train models using labeled attack scenarios.
- Classify new incoming data (e.g., brute force, port scan, malware download, bot vs. human).

Outputs

- Attack labels and classification metrics.

Functional Requirements

ID	Description
FR-8.1	The system shall train ML models for attack classification.
FR-8.2	The system shall classify brute force attacks, port scans, malware downloads, and bot/human behavior.
FR-8.3	The system shall evaluate classifier accuracy using F1-score.
FR-8.4	The system shall aim for classification F1-score ≥ 80%.

3.9 Time-Series Storage of Analysis Results (InfluxDB)

Description

Final analysis outputs are stored in a time-series database for visualization.

Inputs

- Tunneling decisions
- Attack classifications
- Time-based analysis results

Process / Behavior

- Store results with timestamps in InfluxDB.

Outputs

- Time-series datasets ready for dashboards.

Functional Requirements

ID	Description
FR-9.1	The system shall store all analysis results in InfluxDB.
FR-9.2	Each stored entry shall contain timestamps and associated metrics.
FR-9.3	The system shall support queries required by Grafana/Superset dashboards.

3.10 Visualization and Reporting

Description

The system provides visual dashboards for all analysis results.

Inputs

- Time-series data from InfluxDB.

Process / Behavior

- Grafana/Superset connects to InfluxDB and generates visual panels.

Outputs

- Visual dashboards showing tunneling detections, attack classifications, traffic behavior, geographic origins, etc.

Functional Requirements

ID	Description
FR-10.1	The system shall provide visual dashboards for tunneling and attack results.
FR-10.2	The system shall display attack timestamps, geographic origins, and bot/human
FR-10.3	The system shall allow comparison of performance metrics (accuracy, F1-score, false positive rate).
FR-10.4	The system shall generate visual summaries of all final results.

4. External Interface Requirements

4.1 User Interfaces

The system includes a visual interface for analyzing results after statistical and machine-learning outputs are generated.

Dashboard Interfaces

- **Grafana** or **Apache Superset** dashboards are used to visualize system outputs.
- These dashboards display:
    - DNS/HTTP tunneling detections
    - Classified cyber attack types
    - Behavioral distinctions (bot vs. human)
    - Geographical origin of attacks
    - Time-based attack distributions
    - Accuracy, false positive rate, F1-score

Characteristics

- Accessible through a web browser from the central analysis machine or connected network.
- Provides time-series graphs, tables, and interactive panels.
- Supports timestamp-based filtering and comparison of metrics.

ID	Description
UI-1	The system shall provide a web-based graphical interface for visualizing all analysis results.
UI-2	The dashboard shall display both tunneling detection outputs and machine-learning classification outputs
UI-3	The interface shall support time-based filtering and multi-metric comparison.

4.2 Hardware Interfaces

Raspberry Pi 4 Model B

- Runs Cowrie, Honeytrap, and Suricata.
- Exposed to the Internet via DMZ and guest network.
- Sends logs to the central server via MQTT.

Home/Office Router

- Provides DMZ and Guest Network configuration.
- Connects Raspberry Pi directly to external networks to capture real attacks.

Central Analysis Machine

- Receives MQTT log packets.
- Runs SQLite, Python processing, InfluxDB, Grafana/Superset.

ID	Description
HWI-1	The system shall interface with a Raspberry Pi 4 as its primary edge device.
HWI-2	The system shall rely on a router capable of DMZ/Guest Network configuration.
HWI-3	The Raspberry Pi shall interface with the central analysis computer via standard network connectivity.

4.3 Software Interfaces

Honeypot and IDS Tools

- **Cowrie**: SSH interaction logging
- **Honeytrap**: HTTP/FTP service emulation

- **Suricata**: Network packet inspection and alert generation

Data Processing Tools

- **MQTT Broker**: Receives and forwards logs
- **SQLite**: Stores raw logs and extracted features
- **Python (Pandas, Scikit-learn)**:
    - preprocessing
    - feature extraction
    - entropy/sliding-window analysis
    - ML classification

Time-Series & Visualization Software

- **InfluxDB**: Time-series storage
- **Grafana / Apache Superset**: Visualization panels

ID	Description
SW-1	The system shall interface with Cowrie, Honeytrap, and Suricata for log generation.
SW-2	The system shall interface with an MQTT broker for data transfer.
SW-3	The system shall store logs and features using SQLite.
SW-4	The system shall use InfluxDB for storing analysis results.
SW-5	The system shall use Grafana or Superset dashboards for visualization.

4.4 Communication Interfaces

MQTT Communication

- Used to transmit logs from Raspberry Pi to central computer.
- Lightweight, suitable for IoT devices.
- Handles small, frequent log messages.

Network Exposure

- Raspberry Pi is exposed to the Internet through DMZ/Guest Network.
- Allows real attack traffic to reach the honeypots.

Database Interaction

- Local SQLite queries used during preprocessing and feature extraction.
- InfluxDB queries used for visualization.

ID	Description
COM-1	The system shall use the MQTT protocol for log transmission.
COM-2	The Raspberry Pi shall communicate with the central analysis server over a standard IP network.
COM-3	The system shall allow honeypots and IDS tools to receive external attacks via Internet-exposed DMZ/Guest Network connections.
COM-4	The system shall support time-series queries from InfluxDB for dashboard rendering.

5. Non-Functional Requirements

5.1 Performance Requirements

The system must meet performance expectations for tunneling detection, attack classification, and data processing.

Performance Requirements

ID	Description
PR-1	The system shall detect DNS and HTTP tunneling activities with an accuracy of at least 80%.
PR-2	The machine-learning classifiers shall achieve an F1-score of ≥ 80% for brute force, port scanning, malware download attempts, and bot/human classification.
PR-3	The system shall compute entropy and sliding-window metrics efficiently enough to support continuous log analysis
PR-4	The system shall be able to store and process all logs generated during the project timeline without exceeding SQLite's capacity.
PR-5	The system shall load and visualize time-series data from InfluxDB with minimal delay suitable for interactive dashboards.

5.2 Reliability Requirements

The system must reliably capture, store, and analyze real attack traffic.

ID	Description
RL-1	The honeypots (Cowrie, Honeytrap, Suricata) shall operate continuously during the data-collection period unless intentionally stopped.
RL-2	MQTT log transmission shall reliably deliver logs from the Raspberry Pi to the central server without data loss under normal conditions.
RL-3	The system shall ensure that all logs received through MQTT are stored in SQLite without corruption.
RL-4	In case real attack traffic is insufficient, the system shall allow integration of external public datasets (CTU-13, CIC-IDS 2017), as specified in the risk-management plan.

5.3 Security Requirements

The project involves real attack exposure; therefore, security constraints are derived from the project text.

ID	Description
SEC-1	The Raspberry Pi shall remain isolated from production networks by operating only through the DMZ and guest network.
SEC-2	No part of the honeypot system shall be connected to any internal or sensitive university network
SEC-3	All logs and analysis data stored in SQLite and InfluxDB shall remain accessible only within the analysis environment.
SEC-4	The system shall ensure that honeypots do not relay or forward attacks to other systems (decoy only).
SEC-5	The setup shall follow the project's safety requirement of controlled exposure to the internet.

5.4 Usability Requirements

These requirements ensure that researchers and analysts can easily interact with the system.

ID	Description
US-1	The Grafana or Superset dashboards shall present tunneling detections, attack classifications, and performance metrics in a clear and readable format.
US-2	The dashboard interface shall support timestamp-based filtering for detailed analysis.
US-3	All visualization panels shall clearly differentiate between tunneling vs. normal traffic and different attack types.
US-4	The system shall provide visual representation of countries of origin and bot vs. human behavior.

5.5 Maintainability Requirements

The system should be maintainable throughout the 8-month project timeline.

ID	Description
MT-1	Honeypot, MQTT, and database configurations shall be modular to allow updates or resets without affecting the entire system.
MT-2	Python-based preprocessing and machine-learning scripts shall be structured to allow modification and re-testing.
MT-3	The system shall log errors (e.g., failed MQTT transmission, corrupted logs) to support debugging.
MT-4	The code, dataset, and documentation shall be published as open-source on GitHub at project completion.

5.6 Extensibility Requirements

According to our project's long-term vision:

ID	Description
EX-1	The system shall be designed so it can incorporate additional honeypots or IDS tools in the future.
EX-2	The system shall allow replacing or adding ML models for improved attack classification.
EX-3	The time-series storage and dashboard design shall allow adding new metrics (e.g., additional traffic features).

5.7 Portability Requirements

ID	Description
PT-1	The analysis environment (Python + SQLite + InfluxDB + Grafana/Superset) shall be able to run on any compatible Linux-based central server.
PT-2	The codebase shall be platform-independent, allowing migration to different machines if needed.
PT-3	The Raspberry Pi honeypot configuration shall be reproducible on similar low-resource IoT hardware.

6. Other Requirements

6.1 Ethical and Privacy Considerations

The project collects real attack traffic by exposing a Raspberry Pi device to the Internet through a DMZ and guest network. Because the system does not interact with legitimate users or private data, but only records malicious activities initiated by external attackers, the ethical considerations are limited to:

ID	Description
ETH-1	The system shall record only traffic directed at the honeypot device and shall not collect or process any personal, private, or unrelated user data.
ETH-2	The system shall ensure that collected attack logs are used strictly for research, analysis, and academic purposes.
ETH-3	The system shall treat all collected attack data as research material and shall not publish any sensitive information beyond what is allowed in open-source datasets (e.g., anonymized logs).
ETH-4	The system shall ensure safe handling of attack samples (e.g., malware download attempts) without allowing them to execute or spread.

6.2 Monitoring and Logging Requirements

Monitoring and logging are key components of the project, as the goal is to collect high-value real-world attack data.

ID	Description
MLR-1	The system shall continuously monitor SSH, HTTP/FTP, and network traffic through Cowrie, Honeytrap, and Suricata.
MLR-2	All logs produced by honeypots and IDS tools shall be transmitted via MQTT to the central analysis computer.
MLR-3	The system shall store all logs, extracted features, and detection/classification outputs.
MLR-4	The system shall log performance evaluation metrics such as accuracy, false positive rate, and F1-score.
MLR-5	The system shall log time-based information so that attacks and tunneling events can be visualized in dashboards (e.g., timestamps, sequences, frequency of events).

6.3 Data Handling and Output Requirements

The project generates several categories of data: raw logs, preprocessed datasets, extracted features, tunneling detection output, attack classification output, and metrics.

Raw Log Handling

DH	Description
DH-1	The system shall store raw logs from Cowrie, Honeytrap, and Suricata in SQLite without modification.

Feature and Preprocessing Data

ID	Description
DH-2	Extracted features (entropy, URI length, query frequency, etc.) shall be stored and kept available for analysis.

Analysis Output

ID	Description
DH-3	The system shall store tunneling-detection results and attack-classification results in InfluxDB for time-series visualization.
DH-4	The system shall ensure that results can be displayed by Grafana or Apache Superset.

Dataset Publication

The project explicitly states that the dataset and code will be shared openly at the end.

ID	Description
DH-5	At project completion, the system shall provide exportable datasets and analysis code suitable for publication as open-source material on GitHub.
DH-6	Shared datasets shall include real attack logs and extracted features, provided that no sensitive or harmful information is released.

7. Appendices

7.1 Appendix A — Glossary

This glossary summarizes the key technical terms used throughout this SRS:

- **Anomaly Detection:** Identifying patterns that deviate from expected system behavior.
- **Attack Classification:** Categorizing detected events (e.g., brute force, port scanning, malware activity, bot/human behavior).
- **Cowrie:** An SSH/Telnet honeypot that logs attacker interactions.
- **DMZ (Demilitarized Zone):** A network segment exposing the Raspberry Pi device to the Internet while isolating it from internal networks.
- **DNS/HTTP Tunneling:** Hiding data within DNS or HTTP traffic to bypass network protections.
- **Entropy (Shannon Entropy):** A randomness metric used to detect protocol tunneling.

- **Feature:** A numerical attribute extracted from logs, such as entropy, URI length, or query frequency.
- **Grafana:** A visualization tool used for time-series dashboards.
- **Honeytrap:** A low-interaction honeypot simulating HTTP/FTP services.
- **InfluxDB:** A time-series database used to store analysis outputs.
- **IoT (Internet of Things):** A network of connected, resource-limited devices.
- **Machine Learning (ML):** Algorithms used to detect or classify attack behaviors.
- **MQTT:** A lightweight publish/subscribe protocol used for log transmission.
- **SQLite:** A lightweight database storing logs and extracted features.
- **Sliding Window:** A technique for evaluating entropy across sequential time-based segments.
- **Suricata:** A network IDS generating alerts and packet-level logs.
- **Time-Series Database:** A database optimized for storing timestamped data.

7.2 Appendix B — Feature List

The system extracts various measurable characteristics from the log data:

Independent Variables (Features)

- **Session Duration**
- **Query Frequency**
- **Character Entropy**
- **URI Length**
- **IP Address Diversity**
- **Connection Intensity**

Dependent Variables

- Attack Type (e.g., brute force, tunneling, port scanning)
- Behavior Class (e.g., bot vs. human)

These features support both entropy-based tunneling detection and machine-learning-based attack classification.

7.3 Appendix C — Project Schedule Overview

Project timeline:

Time Period	Major Activities
Oct–Nov 2025	Honeypot installation, DMZ exposure, log generation
Nov–Dec 2025	MQTT setup, SQLite schema creation, log ingestion
Dec 2025–Jan 2026	Feature engineering (entropy, session duration, URI, etc.)
Feb–Mar 2026	Statistical tunneling detection model
Mar–Apr 2026	Machine-learning model training & attack classification
Apr–May 2026	Evaluation (accuracy, F1-score, false positive rate), dashboard creation
May 2026	Dataset & code publication, final reporting

7.4 Appendix D — System Architecture Diagram
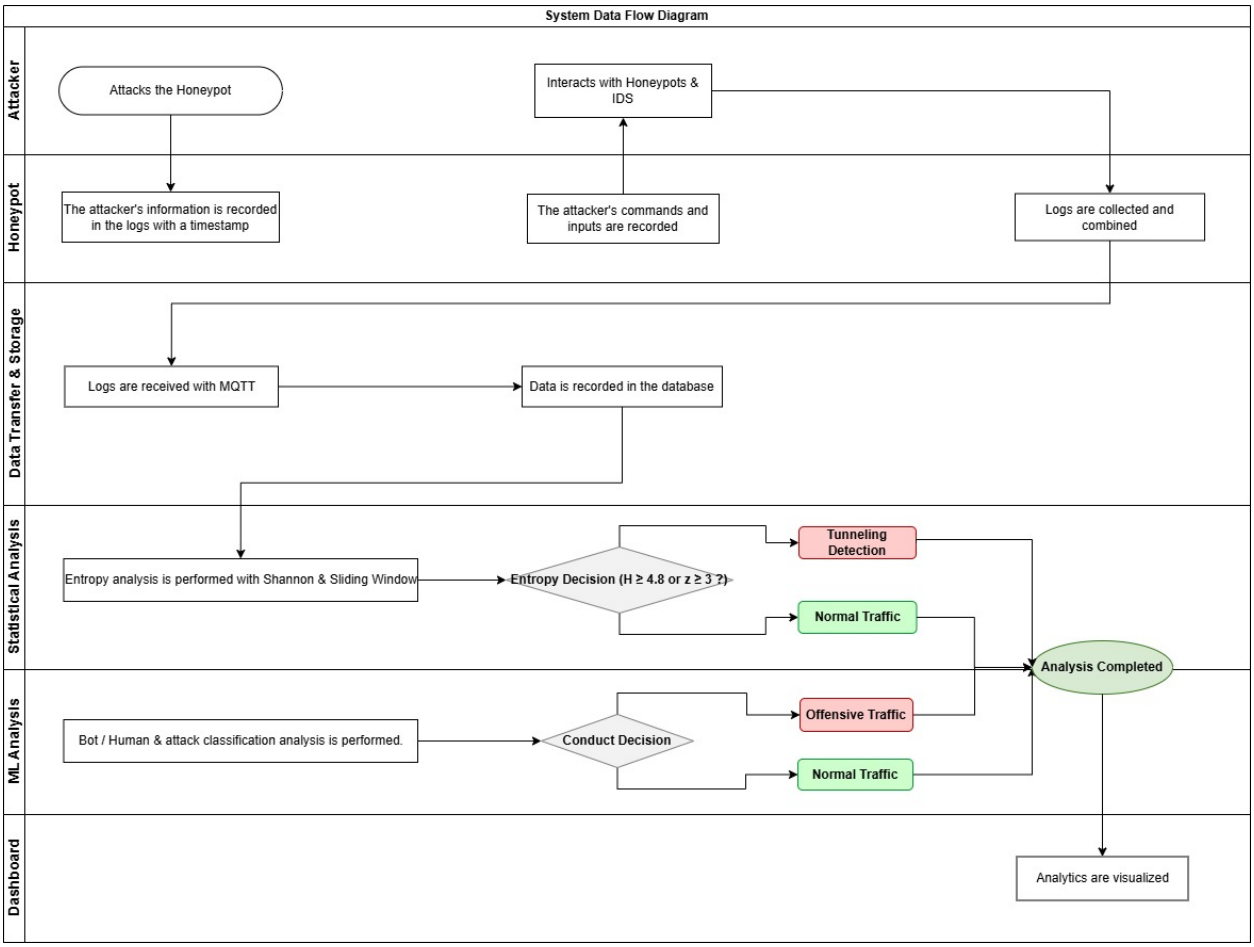
Figure F-1. System Data Flow Diagram